Description

The development of software often relies on the division of labor and collaboration within a team. In the development team, there are various positions such as front-end, back-end, tester, product manager, etc. They each have their own responsibilities and work closely together and communicate frequently to achieve major projects.

In this task, we will use the knowledge of OOP to simulate a simple software development team, which mainly includes two roles: programmer `Coder` and tester `Tester`. Both of them are implemented by a C++ class and inherit from the base class `Agent`. Now, we have implemented `main.cpp`, `Makefile`, and `agent.h` (including declarations of the `Agent` base class and definitions of most interfaces), which can be downloaded from here. We need everyone to write the following three files:

`Agent.cpp`: In this file, you need to complete the interfaces that are not defined by `agent.h`, that is `static Agent::Agent* create_agent(int agent_type, int id, string& codes)`. The `agent_type` is a 0-1 integer, representing the type of agent to be created. 0 is the programmer and 1 is the tester; `id` is the agent's identifier; `codes` is a C++ reference to a string. In this task, all agents' codes point to the same string, which is the code string jointly written by the team.

`Coder.h`: In this file, you need to complete the declaration and definition of the `Coder` class and its interface:

- The information required for the construction of `Coder` is consistent with the base class `Agent`, namely `int id` and `string& codes`, which are the agent's identifier and the C++ reference to the code string jointly written by all agents;
- The `action` interface of `Coder` should accept a string of code from standard input (in this task, an English string without spaces) and add it to the end of the string referred to by `string& codes`;
- The `report` interface of `Coder` should output the string `Coder {agent_id}: {char_comed} characters coded`, where `{agent_id}` and `{char_comed}` are the agent identifier and the total number of characters in the code "written" by the current Coder in the `action` interface.

`Tester.h`: In this file, you need to complete the declaration and definition of the `Tester` class and its interface:

- The information required for constructing the `Tester` should include, in addition to the two already present in the base class `Agent`, a character `char marker` and a 0-1 integer `int flag`, both of which are necessary for performing the test;
- The `action` interface of the `Tester` will perform testing. The specific process is to calculate how many `marker` characters are contained in the current code `string& codes`. If the parity of the number of characters is the same as `flag`, the test will pass, otherwise it will not pass;
- The `report` interface of `Tester` should output the string `Tester {agent_id}: {test passed} tests passed`, where `{agent_id}` and `{test passed}` are the agent identifier and the number of times the current Tester has passed the tests in the `action` interface.

Input format

The first line consists of two integers, `n` and `m`.

For the following `n` lines, each line starts with an integer, which can be divided into two situations:

- If the integer at the beginning of the line is 0, create a programmer `Coder` and the line has no other information
- If the integer at the beginning of the line is 1, create a tester `Tester`, and the line will have two additional inputs, namely the character `marker` and the 0-1 integer `flag`

The following `m` lines start with two integers, the first integer being `agent_id` and the second integer being 0 or 1:

- If the second integer is 0, the `action` interface of the agent with the identifier `agent_id` will be executed. On this basis, if the agent is a programmer, there will be an additional string input in this line, namely the program string "written" by the programmer in this operation
- If the second integer is 1, execute the `report` interface of the agent with the identifier `agent_id`

Output format

The `report` interface output of each agent, with each piece of information occupying one line

Data scale

$1 \leq n \leq 100,\ 1 \leq m \leq 1000$

In the `action` operation of a programmer, the program string is a lowercase English letter string with a length not exceeding 10

Input/output examples

Input (link) and output (link)

Submission format

- You need to submit three files, including `agent.cpp`, `coder.h`, and `tester.h`
- You should package your files into a zip file and upload it. Note: Your files should be located in the root directory of the compressed file, not in a subfolders of the compressed file; When evaluating, OJ will paste the provided `main.cpp`, `Makefile`, and `agent.h` into your directory for compilation and execution